Taller: memoria con límite (35 min)

1. **Aislamiento:** B no guarda lo de A.
2. **Charla larga:** cinco turnos, mirá los tokens.
3. **Recortá** con `trim_messages`: ¿qué olvida?

Práctica M5, pasos 4 a 6. Para casa: ¿recortar o resumir?

LlamaIndex hace solo lo que en v0.2 escribiste



La demo muestra qué decidió por vos.

El mismo RAG en LlamaIndex son tres líneas

```
docs = SimpleDirectoryReader("corpus").load_data()          # leer
indice = VectorStoreIndex.from_documents(docs)              # partir y vectorizar
r = indice.as_query_engine(similarity_top_k=4).query(p)     # buscar y responder
```

- Tu `v02_rag.py` hace lo mismo a la vista.
- Ojo: de fábrica usan OpenAI y leen todo `corpus/`, licencia incluida. La demo antes les dice qué modelo y qué archivos usar.

LlamaIndex respondió largo y sin citar

```
Pregunta: ¿Qué hace interrupt en LangGraph?

── Tu v0.2 · responder (2.3 s)
`interrupt` pausa la ejecución de un grafo en un nodo … y espera una entrada
externa antes de continuar [langgraph__interrupts]. Al llamarlo, … guarda el estado

── LlamaIndex (35.3 s)
Basándose en la información provista, la función `interrupt()` … permite pausar
… 1. **Pausa la ejecución** … 2. **Guarda el estado** … 3. … 4. …   (sin citas)
fuentes: […] · fragmentos: 50
```

- Probalo: cuaderno en vivo de la clase 5, sección 1
- Los 35 s incluyen rehacer el índice. v0.2 lo reusa.

Lo que el framework decide por vos

	v0.2	LlamaIndex, de fábrica
Fragmento	1000 caracteres	1024 tokens
k	4	2
Prompt	Tu `SISTEMA` : citar	El suyo, oculto
Índice	En `.chroma`	Se rehace cada vez

Todo se cambia, pero primero hay que saber que existe.

Comparemos con una pregunta tuya

- Dictá una pregunta de tu tema: la corro en los dos.
- Anotá de cada uno: largo, si cita, tiempo y fuentes.
- ¿Cuál dejarías en tu proyecto, y por qué?

Repaso: hoy te llevás tres ideas



Framework

piezas armadas, pasos escondidos



Memoria

reenviar la conversación



Medir

igual, con cualquier librería

Un hilo por persona: cada conversación guarda solo lo suyo.

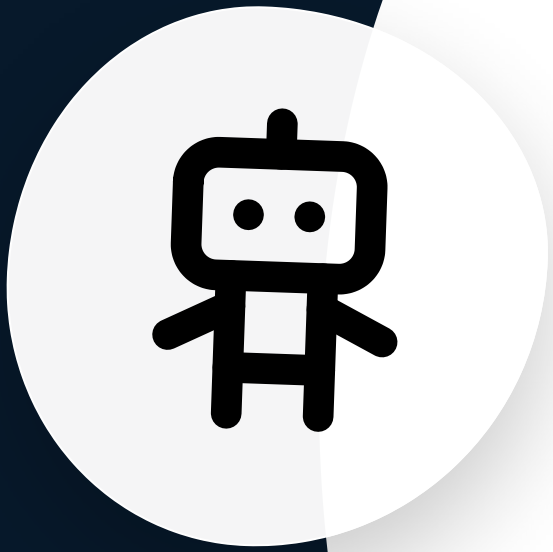
¿Qué ya no podrías explicar?

Nombrá una parte de tu sistema que ahora hace LangChain y que no sabrías explicar sin leer su código.

Anotá tu **confianza (1 a 4)**.

Agentes

M6: v0.4, Agentes



Hoy: el recorrido

De pasos fijos a un modelo que elige



Venís de v0.3: siempre busca y responde. Hoy le das herramientas y lo dejás elegir.

Hoy cada ejemplo corre en vivo: cuaderno de la clase 5.

Primero solo, después lo discutimos

- Control de lectura individual, 5 min
- Discusión de las respuestas, 5 min

Sin asistentes de IA. 5 preguntas sobre las lecturas: estado, patrones e interrupt. En la discusión, las mismas.

Sin mirar: ¿qué hace siempre tu v0.3?



?

¿Qué pasos?

sigue tu v0.3, siempre en el mismo orden



?

¿Otro camino?

una pregunta tuya que necesite la web o una cuenta

Respondé en una línea, sin mirar.

¿Qué hace siempre tu `v0.3` ?

Respuestas: pasos fijos o elige el modelo

Siempre igual

buscar, armar el contexto, responder

Otro camino

algo reciente (la web) o una cuenta exacta

Hoy

que el modelo elija el camino: eso es un agente

¿Para qué te sirve? Que elija la herramienta justa

Le preguntás	Usa
«¿Cómo se calcula el churn?»	Tus documentos
«¿Cuánto creció de marzo a abril?»	La calculadora
«¿A cuánto está hoy el dólar?»	Internet
Lo mismo, con la web caída	Nada: avisa, no inventa

Workflow: pasos fijos. Agente: elige él



Workflow

el camino lo decidís vos · costo predecible · conviene si conocés los pasos

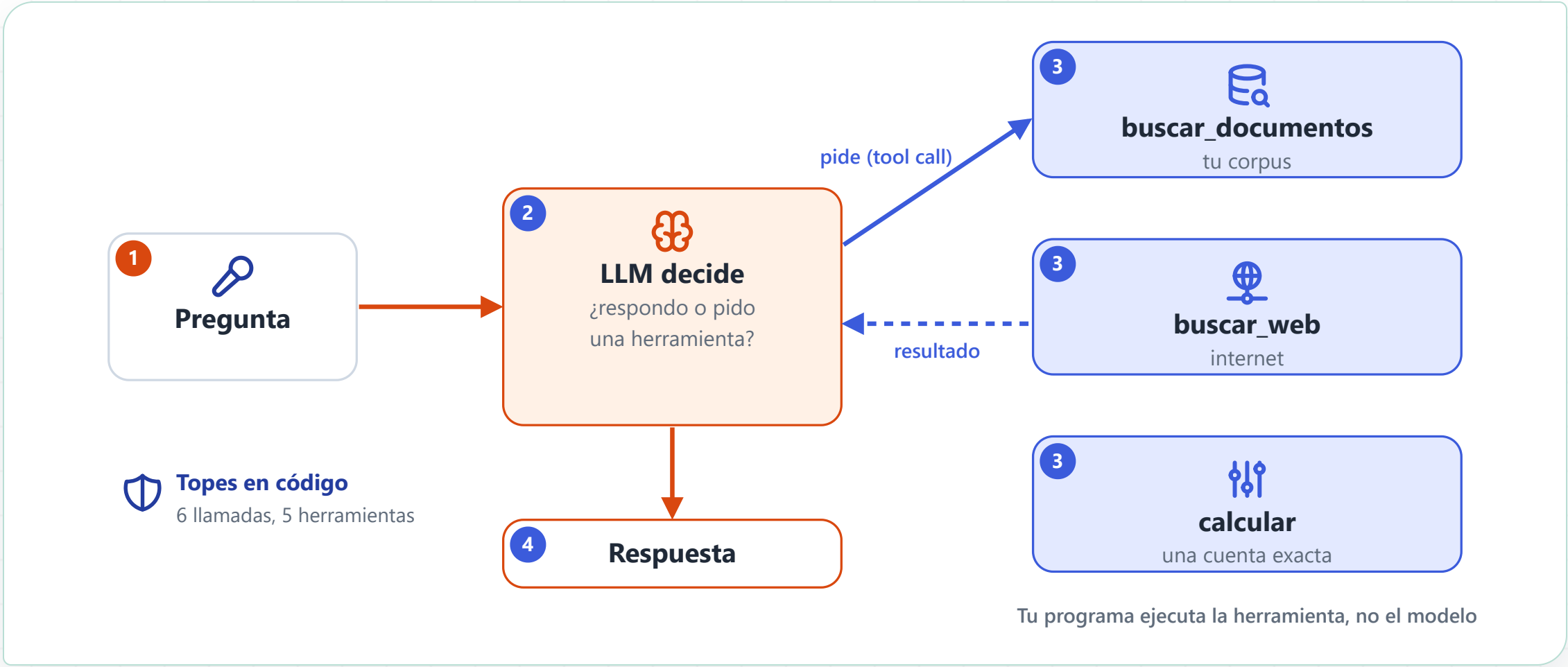


Agente

el camino lo decide el modelo · costo variable · conviene si los pasos cambian

Una receta, o un cocinero que mira la heladera y decide.

Un agente es un LLM dentro de un ciclo



Termina cuando responde o cuando llega al tope.

Las piezas de un agente, con su nombre

Pieza	Qué es
Herramienta (*tool*)	Función que el modelo pide
Pedido (*tool call*)	Nombre y datos; lo corre tu código
Ciclo (*ReAct*)	Piensa, pide, mira y repite
Tope	Cuántas llamadas, como máximo
Instrucciones	Cuándo usar cada herramienta

El modelo solo escribe: la herramienta hace

Solo
cuentas «de cabeza»
no ve tus archivos
no sabe lo de hoy

→

Con herramientas, pide
calcular
buscar_documentos
buscar_web

Herramienta: un botón con cartel. El modelo dice cuál y con qué datos; tu programa lo aprieta.

Una herramienta tiene cuatro partes

Parte	Para qué	En Python	¿La ve?
Nombre	La pide así	`def calcular`	Sí
Descripción	Cuándo usarla	`"""Hace una cuenta…"""`	Sí
Parámetros	Datos y su tipo	`a: float`	Sí
Salida	Qué devuelve	`-> str` , `return`	Después

El código de adentro, nunca.

Una herramienta es una función con etiqueta

```
@tool                                          # la etiqueta: ahora es herramienta
def calcular(                                  # NOMBRE
    operacion: Literal["suma", "resta", "multiplicacion",
                       "division", "porcentaje_cambio"],  # PARÁMETROS y tipos
    a: float, b: float) -> str:                # más parámetros · SALIDA: texto
    """Hace una cuenta con dos números. … Usar para cualquier cuenta."""
    resultado = {"suma": a + b, "multiplicacion": a * b, …}[operacion]
    return f"{resultado:.12g}"                 # adentro: el modelo no lo ve
```

El texto entre `"""` es la **descripción**. `float` = número; `Literal` = opciones cerradas.

El modelo solo ve nombre, tipos y descripción

```
{"name": "calcular",
 "description": "Hace una cuenta con dos números. porcentaje_cambio =
                 (b - a) / a * 100. Usar para cualquier cuenta.",
 "parameters": {"type": "object",
   "properties": {"operacion": {"type": "string", "enum": ["suma", "resta",
                    "multiplicacion", "division", "porcentaje_cambio"]},
                  "a": {"type": "number"}, "b": {"type": "number"}},
   "required": ["operacion", "a", "b"]}}
```

- `name`, `description`, `parameters`: las tres partes que ve.
- Nada del código de adentro: elige leyendo esto.

El modelo pide, tu código ejecuta

```
modelo   AIMessage(content="", tool_calls=[{"name": "calcular",
                  "args": {"a": 5, "b": 3, "operacion": "multiplicacion"}}])
código   calcular(a=5, b=3, operacion="multiplicacion")  →  "15"
vuelve   ToolMessage("15"), y el modelo decide: pide otra o responde
```

- El modelo no ejecuta nada: escribe un pedido.
- Ese ciclo se llama **ReAct**.

Así se ve el ciclo de tu v0.4, mensaje por mensaje

```
[pregunta]     …¿cuántas filas tiene la tabla que compara checkpointer y store?
               Multiplicá ese número por 3.
[modelo pide]  buscar_documentos({'consulta': 'checkpointer store tabla comparar…'})
[tu código]    buscar_documentos → '[langgraph__persistence] ## Checkpointer vs…'
[modelo pide]  calcular({'b': 3, 'operacion': 'multiplicacion', 'a': 5})
[tu código]    calcular → '15'
[respuesta]    La tabla … tiene 5 filas … el resultado es **15**.
```

- Dos herramientas, tres llamadas: dos piden, la tercera responde.
- Gemini 3.5 Flash-Lite; otra corrida: 6 × 3 = 18. En vivo: sección 6.

La descripción vaga no cambió la elección

Alguien dice cuándo
12 de 12: descripción real, o vaga con pistas

=

Nadie dice cuándo
«Busca información.», SISTEMA neutro
11 y 12 de 12

gpt-5.4-nano, dos corridas: diferencia por azar. En vivo: sección 7.

Ejecutar código: poderoso y riesgoso

La calculadora

lo que usa tu v0.4: solo hace cuentas, con datos que tu código valida

Ejecutar código

el modelo escribe Python y se corre: analiza datos y grafica, pero puede hacer cualquier cosa

Ejecutar código pide un entorno aislado y que alguien apruebe.

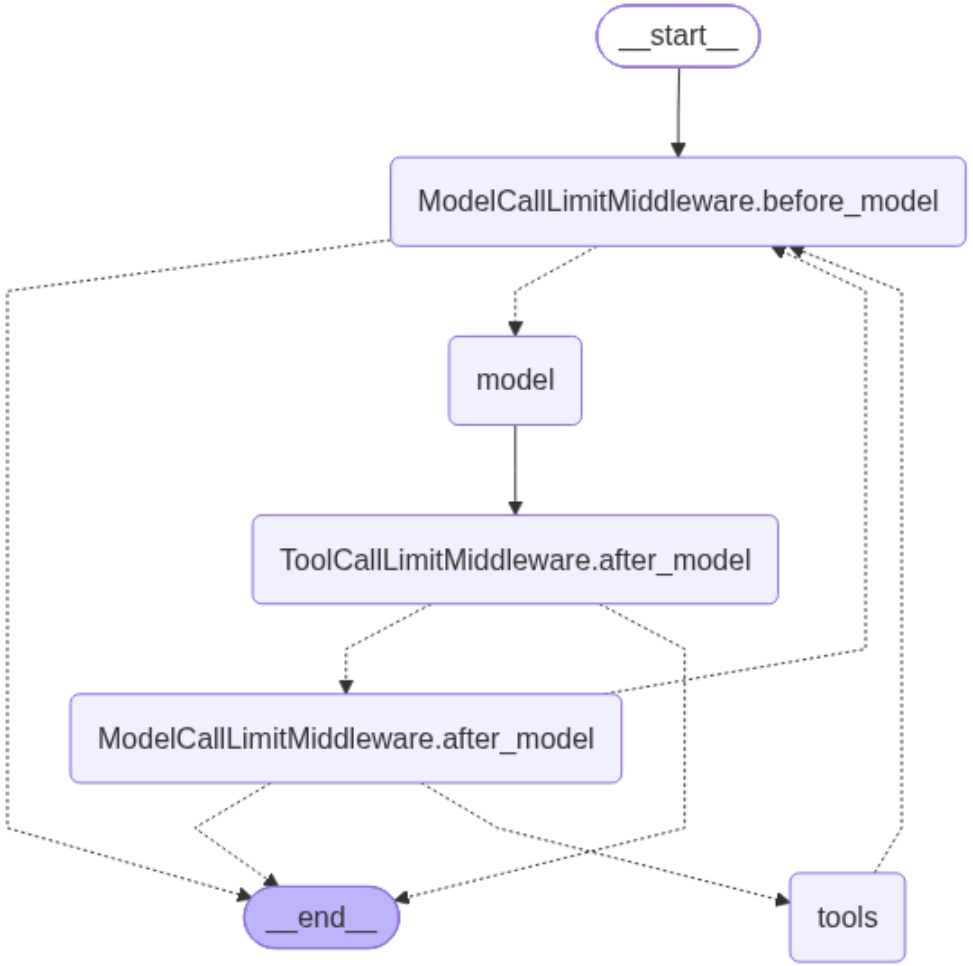
Tu agente de código lo hace: por eso pide permiso.

Los límites van en el código, no en el prompt

```
def calcular(operacion: Literal["suma", "resta", "multiplicacion", "division",
                                "porcentaje_cambio"], a: float, b: float) -> str:
    ...  # opciones cerradas: el modelo elige una, no escribe código
agente = create_agent(modelo, tools=[buscar_documentos, buscar_web, calcular],
                      system_prompt=SISTEMA,
                      middleware=[ModelCallLimitMiddleware(run_limit=6)])
```

- Tope: hasta 6 llamadas por pregunta, aunque el prompt diga otra cosa.
- Herramientas de solo lectura: buscan y calculan, no escriben.

En el grafo real, los topes son cajas del ciclo



Con la web caída, uno inventó y otro avisó

Modelo local chico
«La última versión es 0.1.2»

≠

Gemini 3.5 Flash-Lite (17/10)
«devolvió un error»

- ¿Cuál liberarías?
- ¿Qué haría falta para que el chico avise?
- Pista: ¿pidió la web?

Con la web caída, el ERROR llegó al modelo y lo dijo

```
[pregunta]     ¿Cuál es la última versión de LangGraph publicada en PyPI?
[modelo pide]  buscar_web({'consulta': 'langgraph pypi version'})
[tu código]    buscar_web → 'ERROR: la búsqueda web no respondió. No inventes; …'
[respuesta]    Lo siento, la búsqueda web devolvió un error, por lo que no puedo
               confirmar la última versión exacta de LangGraph…
```

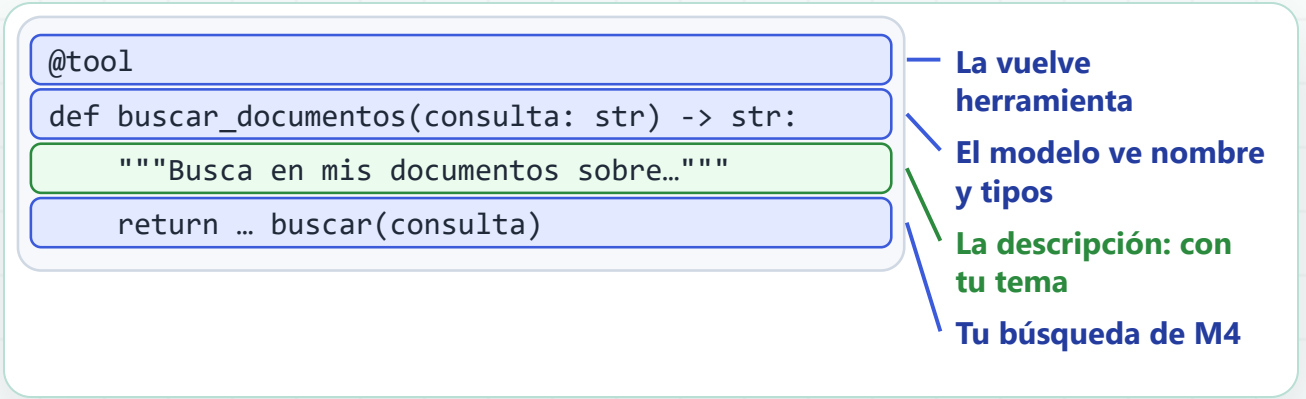
- Gemini 3.5 Flash-Lite leyó el ERROR. En vivo: sección 8.
- Si lo ignora, lo ataja el guardrail (clase 7). Si ni pide la web, no.

Taller: tres herramientas para tu tema (55 min)

1. **Tu descripción** de `buscar_documentos`.
2. **Tu suite de selección:** `datos/mi_seleccion.jsonl`
3. **Predecí** dónde falla.

Práctica M6, arranque y «En clase», pasos 1 y 3.

Primer control · 10–15 min: ves qué herramienta pidió en cada pregunta.



Mismas descripciones, otro modelo: de 9 a 12 de 12

Modelo local chico
eligió bien 9 de 12

→

Gemini 3.5 Flash-Lite
12 de 12, en 3 corridas

- Misma suite, 17/10.
- Antes de culpar a la descripción, probá otro modelo.